L21 — Operations on Doubly Linked List; Complexity Analysis and Applications

Unit: 2 | **Chapter:** Linked Lists | **BT Level:** BT4 | **CO:** CO1, CO2

Learning Objectives

- Implement all DLL operations: search, reverse, and merge
- Analyze time and space complexity of each DLL operation
- Build an LRU Cache using a DLL + hash map
- Compare DLL complexity with array and SLL

---

1. Complete DLL Operations

1.1 Reverse a Doubly Linked List — O(n)

Swapping `prev` and `next` pointers for every node:

```
def reverse(self):
    curr = self.head
    new_head = None

    while curr:
        # Swap prev and next for each node
        curr.prev, curr.next = curr.next, curr.prev
        new_head = curr         # Track the last processed node
        curr = curr.prev        # Move to what was next (now prev after

    self.head = new_head
```

1.2 Find Middle Node — O(n)

```
def find_middle(self):
    slow = self.head
    fast = self.head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

1.3 Sort a Doubly Linked List (Insertion Sort) — $O(n^2)$

```
def insertion_sort_dll(self):
    if not self.head:
        return
    sorted_head = self.head
```

```
    curr = self.head.next

    while curr:
        next_node = curr.next
        key = curr.data
        temp = curr.prev

        # Find correct position in sorted portion
        while temp and temp.data > key:
            temp.next.data = temp.data   # Shift value right
            temp = temp.prev

        if temp:
            temp.next.data = key
        else:
            sorted_head.data = key   # Inserted before sorted_head

        curr = next_node
```

1.4 Remove All Occurrences of a Value — O(n)

```
def remove_all(self, key):
    curr = self.head
    while curr:
        if curr.data == key:
            if curr.prev:
                curr.prev.next = curr.next
            else:
                self.head = curr.next
            if curr.next:
                curr.next.prev = curr.prev
        curr = curr.next
```

1.5 Merge Two Sorted DLLs — O(m + n)

```
def merge_sorted_dlls(head1, head2):
    dummy = DNode(-1)
    curr = dummy

    while head1 and head2:
        if head1.data <= head2.data:
            curr.next = head1
            head1.prev = curr
            head1 = head1.next
        else:
            curr.next = head2
            head2.prev = curr
            head2 = head2.next
        curr = curr.next

    if head1:
        curr.next = head1
        head1.prev = curr
```

```
    if head2:
        curr.next = head2
        head2.prev = curr

    result_head = dummy.next
    if result_head:
        result_head.prev = None
    return result_head
```

---

2. Complexity Summary

Operation	SLL	DLL	Array
Access by index	O(n)	O(n)	O(1)
Insert at beginning	O(1)	O(1)	O(n)
Insert at end	O(n)	O(n) [O(1) w/ tail]	O(1)
Insert at given node	O(1)*	O(1)	O(n)
Delete at beginning	O(1)	O(1)	O(n)
Delete at end	O(n)	O(n) [O(1) w/ tail]	O(1)
Delete given node	O(n)	**O(1)**	O(n)
Search	O(n)	O(n)	O(n)
Reverse	O(n)	O(n)	O(n)
Memory per element	data + 1 ptr	data + 2 ptrs	data only

* SLL insert at given node: O(1) only if you have the previous node reference.

---

3. Real-World Application: LRU Cache

An **LRU (Least Recently Used) Cache** evicts the least recently accessed item when capacity is full.

Implementation: DLL + Hash Map → O(1) for both get and put

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = OrderedDict()  # Maintains insertion order

    def get(self, key):
        if key not in self.cache:
            return -1
        # Move to end (most recently used)
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key, value):
```

```
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)  # Remove least recently use
```

Manual implementation using DLL + dict:

```
class LRUCacheManual:
    def __init__(self, capacity):
        self.capacity = capacity
        self.map = {}            # key → DLL node
        # Sentinel nodes for O(1) operations
        self.head = DNode(-1)    # Most recently used end
        self.tail = DNode(-1)    # Least recently used end
        self.head.next = self.tail
        self.tail.prev = self.head

    def _remove(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def _insert_front(self, node):
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.map:
            return -1
        node = self.map[key]
        self._remove(node)
        self._insert_front(node)   # Move to front (most recent)
        return node.val

    def put(self, key, val):
        if key in self.map:
            self._remove(self.map[key])
        node = DNode(val)
        node.key = key
        self._insert_front(node)
        self.map[key] = node
        if len(self.map) > self.capacity:
            lru = self.tail.prev    # Least recently used
            self._remove(lru)
            del self.map[lru.key]
```

Time complexity: O(1) for both `get` and `put`

---

4. Other DLL Applications

Application	Role of DLL
Browser history	Back/forward navigation (doubly linked)
Text editors	Undo/redo — cursor movements
Music player	Previous/next track
Memory allocators	Free block list (glibc malloc)
Operating system	Process scheduling doubly linked queues
Deque	Efficient insertion/deletion at both ends

Summary

- DLL operations (insert, delete given node) run in **O(1)** — key advantage over SLL.
- Reverse: swap `prev` and `next` for all nodes → O(n).
- Merge two sorted DLLs → O(m + n).
- **LRU Cache = DLL + Hash Map** → O(1) get and put — a classic system design pattern.
- DLL memory cost: 2 pointers per node (16 bytes on 64-bit) vs 1 pointer for SLL.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.2 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.1 (Springer, 2020)
- LeetCode #146 — LRU Cache